

Loan Approval Prediction – Data Science Project Project Overview This project focuses on predicting loan approval status using machine learning techniques and structured financial datasets. The notebook includes: - Data loading and preprocessing - Exploratory Data Analysis (EDA) - Handling missing values - Feature engineering - Model training and evaluation - Prediction analysis Technologies Used - Python - Pandas - NumPy - Scikit-learn - Matplotlib - Seaborn Objective The objective of this project is to build a predictive machine learning model that can determine whether a loan application is likely to be approved based on applicant information and financial factors.

Step 1: Import Required Libraries

1. Import Packages & Data

Code:

```
!ls
```

Code:

```
import zipfile

with zipfile.ZipFile('archive (2).zip', 'r') as zip_ref:
    zip_ref.extractall('archive2')

print("Done")
```

Code:

```
!ls archive2
```

Code:

```
#Import packages
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn import svm

#Read CSV data
data = pd.read_csv("archive2/loan_prediction.csv")
```

Code:

```
#preview data
data.head()
```

2. Data Quality & Missing Value Assessment

Code:

```
#Preview data information
data.info()
```

Code:

```
#Check missing values
data.isnull().sum()
```

Gender - Missing Values

Code:

```
# percent of missing "Gender"
print('Percent of missing "Gender" records is %.2f%%' %((data['Gender'].isnull().sum()/data.shape[0])*100))
```

Code:

```
print("Number of people who take a loan group by gender :")
print(data['Gender'].value_counts())
sns.countplot(x='Gender', data=data, palette = 'Set2')
```

Married - Missing Values

Code:

```
# percent of missing "Married"
print('Percent of missing "Married" records is %.2f%%' %((data['Married'].isnull().sum()/data.shape[0])*100))
```

Code:

```
print("Number of people who take a loan group by marital status :")
print(data['Married'].value_counts())
sns.countplot(x='Married', data=data, palette = 'Set2')
```

Dependents- Missing Values

Code:

```
# percent of missing "Dependents"
print('Percent of missing "Dependents" records is %.2f%%' %((data['Dependents'].isnull().sum()/data.shape[0])*100))
```

Code:

```
print("Number of people who take a loan group by dependents :")
print(data['Dependents'].value_counts())
sns.countplot(x='Dependents', data=data, palette = 'Set2')
```

Self Employed - Missing Values

Code:

```
# percent of missing "Self_Employed"
print('Percent of missing "Self_Employed" records is %.2f%%' %((data['Self_Employed'].isnull().sum()/data.shape[0])*100))
```

Code:

```
print("Number of people who take a loan group by self employed :")
print(data['Self_Employed'].value_counts())
sns.countplot(x='Self_Employed', data=data, palette = 'Set2')
```

Loan Amount - Missing Values

Code:

```
# percent of missing "LoanAmount"
print('Percent of missing "LoanAmount" records is %.2f%%' %((data['LoanAmount'].isnull().sum()/data.shape[0])*100))
```

Code:

```
ax = data["LoanAmount"].hist(density=True, stacked=True, color='teal', alpha=0.6)
data["LoanAmount"].plot(kind='density', color='teal')
ax.set(xlabel='Loan Amount')
plt.show()
```

Loan Amount Term - Missing Values

Code:

```
# percent of missing "Loan_Amount_Term"
print('Percent of missing "Loan_Amount_Term" records is %.2f%%' %((data['Loan_Amount_Term'].isnull().sum()/data.shape[0])*100))
```

Code:

```
print("Number of people who take a loan group by loan amount term :")
print(data['Loan_Amount_Term'].value_counts())
sns.countplot(x='Loan_Amount_Term', data=data, palette = 'Set2')
```

Credit History - Missing Values

Code:

```
# percent of missing "Credit_History"
print('Percent of missing "Credit_History" records is %.2f%%' %((data['Credit_History'].isnull().sum()/data.shape[0])*100))
```

Code:

```
print("Number of people who take a loan group by credit history :")
print(data['Credit_History'].value_counts())
sns.countplot(x='Credit_History', data=data, palette = 'Set2')
```

3. Final Adjustments to Data

Based on my assessment of the missing values in the dataset, I'll make the following changes to the data:

- * If "Gender" is missing for a given row, I'll impute with Male (most common answer).
- * If "Married" is missing for a given row, I'll impute with yes (most common answer).
- * If "Dependents" is missing for a given row, I'll impute with 0 (most common answer).
- * If "Self_Employed" is missing for a given row, I'll impute with no (most common answer).
- * If "LoanAmount" is missing for a given row, I'll impute with mean of data.
- * If "Loan_Amount_Term" is missing for a given row, I'll impute with 360 (most common answer).
- * If "Credit_History" is missing for a given row, I'll impute with 1.0 (most common answer).

Code:

```
train_data = data.copy()
train_data['Gender'].fillna(train_data['Gender'].value_counts().idxmax(), inplace=True)
train_data['Married'].fillna(train_data['Married'].value_counts().idxmax(), inplace=True)
train_data['Dependents'].fillna(train_data['Dependents'].value_counts().idxmax(), inplace=True)
train_data['Self_Employed'].fillna(train_data['Self_Employed'].value_counts().idxmax(), inplace=True)
train_data["LoanAmount"].fillna(train_data["LoanAmount"].mean(skipna=True), inplace=True)
train_data['Loan_Amount_Term'].fillna(train_data['Loan_Amount_Term'].value_counts().idxmax(), inplace=True)
train_data['Credit_History'].fillna(train_data['Credit_History'].value_counts().idxmax(), inplace=True)
```

Code:

```
#Check missing values
train_data.isnull().sum()
train_data
```

Code:

```
#Convert some object data type to int64
gender_stat = {"Female": 0, "Male": 1}
yes_no_stat = {'No' : 0, 'Yes' : 1}
dependents_stat = {'0':0, '1':1, '2':2, '3+':3}
education_stat = {'Not Graduate' : 0, 'Graduate' : 1}
property_stat = {'Semiurban' : 0, 'Urban' : 1, 'Rural' : 2}

train_data['Gender'] = train_data['Gender'].replace(gender_stat)
train_data['Married'] = train_data['Married'].replace(yes_no_stat)
train_data['Dependents'] = train_data['Dependents'].replace(dependents_stat)
train_data['Education'] = train_data['Education'].replace(education_stat)
train_data['Self_Employed'] = train_data['Self_Employed'].replace(yes_no_stat)
train_data['Property_Area'] = train_data['Property_Area'].replace(property_stat)
```

Code:

```
#Preview data information
data.info()
data.isnull().sum()
```

4. Making Prediction

Code:

```
#Separate feature and target
x = train_data.iloc[:,1:12]
y = train_data.iloc[:,12]

#make variabel for save the result and to show it
classifier = ('Gradient Boosting', 'Random Forest', 'Decision Tree', 'K-Nearest Neighbor', 'SVM')
```

```
y_pos = np.arange(len(classifier))
score = []
```

Code:

```
clf = GradientBoostingClassifier()
scores = cross_val_score(clf, x, y, cv=5)
score.append(scores.mean())
print('The accuratation of classification is %.2f%%' %(scores.mean()*100))
```

Code:

```
clf = RandomForestClassifier(n_estimators=10)
scores = cross_val_score(clf, x, y, cv=5)
score.append(scores.mean())
print('The accuratation of classification is %.2f%%' %(scores.mean()*100))
```

Code:

```
clf = DecisionTreeClassifier()
scores = cross_val_score(clf, x, y, cv=5)
score.append(scores.mean())
print('The accuratation of classification is %.2f%%' %(scores.mean()*100))
```

Code:

```
clf = KNeighborsClassifier()
scores = cross_val_score(clf, x, y, cv=5)
score.append(scores.mean())
print('The accuratation of classification is %.2f%%' %(scores.mean()*100))
```

Code:

```
clf = svm.LinearSVC(max_iter=5000)
scores = cross_val_score(clf, x, y, cv=5)
score.append(scores.mean())
print('The accuratation of classification is %.2f%%' %(scores.mean()*100))
```

5. Result

The result is Gradient Boosting Classifier have the highest score from other classification algorithm. These result are similar to my previous works.

Code:

```
plt.barh(y_pos, score, align='center', alpha=0.5)
plt.yticks(y_pos, classifier)
plt.xlabel('Score')
plt.title('Classification Performance')
plt.show()
```

Reference 1. J. Heo and J. Y. Yang, "AdaBoost Based Bankruptcy Forecasting of Korean Construction Company," Applied Soft Computing, vol. 24, pp. 494-499, 2014. 2. C.-F. Tsai, "Feature Selection in Bankruptcy Prediction," Knowledge Based System, pp. 120-127, 2009.

****Conclusion**** The loan prediction model was successfully developed using machine learning techniques to classify loan approval status. Data preprocessing and feature analysis improved the quality of predictions. The model can assist financial institutions in making faster and more consistent loan approval decisions.

****Recommendations**** 1)Improve data quality by handling missing values and outliers before training the model. 2)Include additional applicant information such as employment stability and repayment history to enhance prediction accuracy. 3)Compare multiple machine learning algorithms and select the model with the best performance metrics. 4)Perform regular model retraining using updated loan application data to maintain accuracy over time. 5)Use the prediction system as a decision-support tool alongside human evaluation to ensure fair and reliable loan approvals.